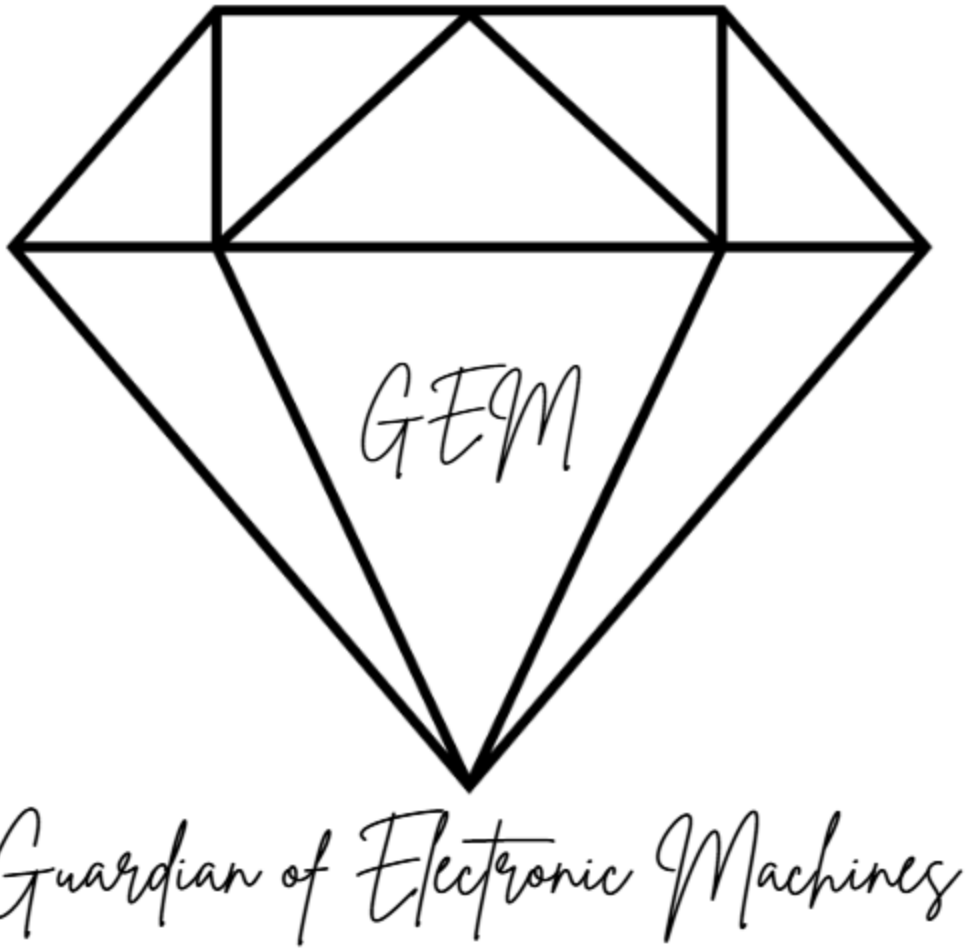

Genevieve McNeil

05/14/2025

Lab 04 - Intrusion Detection

Executive Summary

The goal for this lab was to explore Snort Intrusion Detection System (Snort) IDS within a Windows VM in SimSpace Cyber Range. The lab was separated into 4 separate parts. For Part 1, Locating and Starting SNORT, I located Snort within Win-Hunt-26 VM. I then found the correct network interface and confirmed that Snort started successfully. For Part 2 of the lab, Analysis of SNORT Configuration Settings, I reviewed the SNORT configuration options. I gained a further understanding of how important network interface settings were in regards to SNORT monitoring traffic. For Part 3, Capture and Analyze Network Traffic, I created inbound traffic to the Win-Hunt-26 VM that was running SNORT. I also reviewed the results within Wireshark. For part 4, SNORT Rules, I reviewed Common Vulnerabilities and Exposures CVE-2017-5638 and CVE-2021-44228. I wrote a custom SNORT rule for each CVE and then added those rules to the SNORT file called “local”. I validated my rules by through adding them to SNORT’s local rule set. I then was able to confirm that SNORT started successfully with no setbacks. Altogether, this lab helped me understand how SNORT operated, how to analyze traffic, and how it can be used in network defense. I am grateful for the opportunity to complete this lab and will ensure to keep in mind what I learned as I move forward in my career.

Table of Contents

Part 1: Locating and starting SNORT

In this first part of the lab, I logged into simspace.com. I navigated to the Events section, then to Virtual Machines, and then opened Win-Hunt-26. I decided to begin reviewing files in the VM and located a file named “SNORT” and double clicked. Within the file, there were a variety of additional files saved under SNORT: backup, bin, doc, etc, lib, log, MySnort, preproc_rules, and rules. To be transparent, I was unfamiliar with using SNORT, so I decided to conduct a few minutes of research in order to complete the lab properly.

I first started with inputting “snort” to which I received an error: “'snort' is not recognized as an internal or external command, operable program or batch file.” I also inputted “snort -W” and received the same error once again. These results did confuse me, as I fully believed that the commands would work correctly. I then reached out to another classmate in order to get advice on what commands worked for them. I then was advised that I need to change the directory to: \Users\Administrator\Desktop\Snort\bin to download SNORT. It worked, as shown in Figure 2. I then was advised to enter “snort -i 3 -c ..\etc\snort.conf -T”, as the results of the interface included an IP address beginning with 127 (it was index 3). Both shown in Figure 2 and 3.

It is extremely important to select the correct interface when using SNORT. This is because selecting the correct interface will allow SNORT to monitor the correct traffic on the network. If by chance the incorrect interface is selected whilst using SNORT, it

will not operate correctly because it will not capture packets. Using the correct interface will allow SNORT to analyze the intended packets for suspicious activity.

IDS stands for “intrusion detection systems” (“IDS vs IPS – Differences between IDS and IPS | Versa Networks”). It is a software created in order to keep a network or system safe from malicious traffic. With IDS, suspicious activity is normally reported to administrator (s) (“IDS vs IPS – Differences between IDS and IPS | Versa Networks”). IPS stands for “intrusion prevention system”. It is also known as “intrusion detection and prevention systems” (IPDS)(“IDS vs IPS – Differences between IDS and IPS | Versa Networks”). This is an application that operates by locating, reporting, and preventing potential malware (“IDS vs IPS – Differences between IDS and IPS | Versa Networks”). IPS mode requires that SNORT be placed directly on the path of the traffic. This is done normally through bridging or redirecting in order for it to be able to actively inspect and block packets in real-time.

SNORT is useful with various features that allow network administrators to monitor systems and detect/track malicious activity. These features include:

1. **Traffic Monitor:** SNORT can be utilized to keep an eye on the traffic that enters and exits a network. It monitors traffic in real-time and sends alerts to users if it notices potentially dangerous packets and/or threats on Internet Protocol (IP) networks (Fortinet).
2. **Packet Logging**: It enables packet logging because of its packet logger mode (Fortinet). The packet logging mode means that it logs packets to the disk, and

SNORT obtains each packet and logs in a directory organized structure based on the IP address of the host network (Fortinet).

3. **Protocol Analysis**: It is able to perform protocol analysis (Fortinet). Protocol analysis is a network sniffing technique that captures data across protocol layers for a more thorough inspection (Fortinet). Because of this, network administrators are able to inspect packets that they suspect as malicious (Fortinet). This is important for analyzing protocols such as the Transmission Control Protocol/IP (TCP/IP) stack (Fortinet).
4. **Contact Matching**: It organizes its rules by protocol (ie: IP or TCP) (Fortinet). It then organizes by the port number and lastly, it organizes the rules by the content it contains (Fortinet). Rules with content use a multi-pattern matcher in order to enhance performance, especially for protocols such as HTTP (Fortinet). Rules without content are always evaluated, which will impact performance negatively (Fortinet).
5. **OS Fingerprinting**: Operating system fingerprinting is based upon the concept that every platform holds a unique TCP/IP stack (Fortinet). This process allows SNORT to determine the OS platform that is being utilized by a system who has access to a network (Fortinet).
6. **Installation opportunities:** SNORT is able to be utilized on all operating systems, which includeLinux and Windows (Fortinet).
7. **Open source:** SNORT is free open-source software. It is available for people to use an Intrusion Detection System (IDS) or Intrusion Prevention System (IPS) to monitor and secure their network (Fortinet).

With the second part of the lab, I will research, review, and interpret the various SNORT configuration options.

Part 2: Analysis of SNORT Configuration Settings

As previously stated, I worked within Win-Hunt-26. In order to view the contents of snort.conf, I entered “type snort.conf” then I entered “type snort.conf > conf_output.txt”. This command pipes the file to a text file for better readability. It automatically opened the file and I was able to read it. This is shown in Figure 5. It is noteworthy to mention some of the rules and what they mean. Please view the information below:

- **include SRULE_PATH\local.rules**: This loads custom rules that were written by the organization or the user themselves (ChatGPT, 2025). This is usually used for testing or specific alerts. (ChatGPT, 2025).
- **include $RULE_PATH\app-detect.rules**: This rule is placed to detect traffic that is related to common applications (ChatGPT, 2025). It helps SNORT find out what apps are being used.
- **include $RULE_PATH\attack-responses.rules**: This rule tries to find responses that were sent by systems that were under attack (ChatGPT, 2025). For example, error messages that could indicate intrusion attempts or buffer overflow attacks. (ChatGPT, 2025).
- **include $RULE_PATH\backdoor.rules**: This rule detects backdoor malware that is attempting to obtain unauthorized remote access to systems. (ChatGPT, 2025).

- **include $RULE_PATH\bad-traffic.rules**: This rule flags any invalid or suspicious network traffic such as traffic that is from reserved IP ranges. (ChatGPT, 2025).
- **include SRULE_PATH\blacklist.rules**: This rule blocks or alerts on traffic that is traveling to or from any known bad IPs, domains, or URLs. (ChatGPT, 2025).
- **include SRULE_PATH botnet-cnc.rules**: This rule detects Command (C2) and Control (CNC) traffic from any infected hosts that are a part of a botnet. (ChatGPT, 2025).
- **include $RULE_PATH\browser-chrome.rules**: This rule tries to locate any attacks that are attempting to target Google Chrome. (ChatGPT, 2025).
- **include $RULE_PATH\browser-firefox.rules**: This rule tries to locate any exploits that are specific to Firefox. (ChatGPT, 2025).
- **include SRULE_PATH \browser-ie.rules**: This rule tries to detect exploits and vulnerabilities. (ChatGPT, 2025).
- **include SRULE_PATH\browser-other.rules**: This rule tries to catch any attacks or exploits on browsers that are less common. (ChatGPT, 2025).
- **include SRULE_PATH\browser-plugins.rules**: This rule detects any malicious activities within plugins such as Java or Flash. (ChatGPT, 2025).
- **include $RULE_PATH\browser-webkit.rules**: This rule detects any attacks launched towards browsers that use the WebKit engine (such as Safari). (ChatGPT, 2025).
- **include SRULE_PATH\chat.rules**: This rule monitors chat applications for data exfiltration or any potential abuse. (ChatGPT, 2025).

- **include $RULE_PATH\ content-replace.rules**: This rule allows SNORT to replace content located in packets. (ChatGPT, 2025).

  As shown in Figure 6, the SNORT file included:

“# Configure DAQ related options for inline operation. For more information, see README.daq

config daq: <type>

config daq_dir: ‹dir›

config daq_mode: ‹mode>

config daq_var: ‹var>”

In order to enable IPS (inline) mode, I would modify “config daq: <type>” to “config daq: af packet” and “config daq_mode: inline”. In part 3, I will send traffic to my Win-hunt-26 VM that is running SNORT. I will ensure to utilize WireShark so that I can analyze the traffic.

Part 3: Capture and Analyze Network Traffic

In order to send traffic to the SNORT VM with an alternative VM, I logged back into SimSpace and reviewed any open Kali-Hunt machines. I checked Kali-Hunt-01 and it looked like someone left their work open on it. I then moved to Kali-Hunt-02. However, that one also looked like it was in use as well. I decided to skip all the way until Kali-Hunt-07, and noticed that that VM had applications left left open as well, as a student had entered the “curl” command to an IP address. Kali-Hunt-27 and Kali-Hunt-29 were both in use as well. I would like to note that none of the VM’s that I opened had active

users, so I thought they were free to use. Finally, I logged into Kali-Hunt-26 and it had not been in use.

I logged back into my Win-Hunt-26 VM and entered “ipconfig” to determine my IP address, which was 172.16.3.30. I then navigated back to my Kali-Hunt-26. I did run into a roadblock here, as Kali-Hunt-26 ended up freezing and would not work after closing and re-opening about 3 times. I refreshed the page, closed the VM, and refreshed Sim Space - it still would not work. So, I decided to move forward and utilize Kali-Hunt-40. I entered “ping 172.16.3.30” but it continued to ping without allowing me to stop it. I decided to work on a different assignment in the meantime while waiting for it to load. I came back to the lab and was able to exit the CLI. Pleae review the ping commands in Figure 8.

In Win-Hunt-26, I changed the directory to \Users\Administrator\Desktop\Snort\bin and then entered “snort -i 3 -c ..\etc\snort.conf -A console” to ensure SNORT was running. Then, I refreshed Kali-Hunt-40 (as it closed within a few minutes once again) and then ran the following commands:

- ping 172.16.3.30 (ICMP)
- telnet 172.16.3.30 23 (Telnet)
- ftp 172.16.3.30 (FTP)
- nmap -sS 172.16.3.30 (TCP scan)
- curl http:\\172.16.3.30

The commands: “curl http:\\172.16.3.30”, “ftp 172.16.3.30”, and “telnet 172.16.3.30 23” were not successful. However, “nmap -sS 172.16.3.30” did work correctly, as well as “ping 172.16.3.30”. I did notice that I had not started SNORT on my

Win-Hunt-26 VM, so I entered “snort -i 3 -c ..\etc\snort.conf -A console”. This did not work, so I ensured I was in the right directory and entered “snort -i 3 -c ..\etc\snort.conf -T” to test - it was successful. This had me stumped for a moment, as I was not sure how to move forward.

I then realized I should attempt to use the command “snort -i 2 -A console -c C:\Users\Administrator\Desktop\Snort\etc\snort.conf”. This is because I wanted to use the IP address on index 2 (10.10.0.35 for Win-Hunt-26). At this point in the lab, Kali-Hunt-40 froze once again. This occurred for about 30 minutes, even after exiting and refreshing multiple times.

I realized some of my issues are because I have been using Kali-Hunt as my second VM. I decided to use Win-Hunt-40 from now on. I had much more success attempting to enter these commands below:

- ping 10.10.0.35 (ICMP)
- telnet 10.10.0.35 23 (Telnet)
- ftp 10.10.0.35 (FTP)
- nmap -sS 10.10.0.35 (TCP scan)
- curl http:\\10.10.0.35\ (HTTP request)

I was able to view the traffic, as shown in Figure 11. As a Blue Teamer, I would state that packets 1-8 are normal and show no signs of a potential attack. However, the presence of multiple types of ICMP, telnet, FTP, TCP, and HTTP requests would be a red flag. Although most of the commands I entered were not successful, it does show that a potential threat actor is attempting to connect to a device on the network. An attacker could simply be testing what computer (s) have open ports and which ones are

available. The ping command was successful and so was the nmap scan. The nmap scan showed that ports 135 and 445 were open, the mac address, and stated that the host that was scanned was Intel. Next, I will review the SNORT rules and created my own as well.

Part 4: SNORT Rules

This is a structure of a SNORT rule: “alert tcp any 21 -> 10.10.35 any (msg: "TCP Packet is detected"; Sid: 1000010” (Cyvatar, 2022). Additionally, the Rule Header is the first part of the rule, and second part is Rule Option. The entire rule structure is as follows:

- Action
- Protocol
- Source
- Address
- Source
- Port
- Direction
- Destination Address
- Destination Port
- And finally, the Rule Option

(Cyvatar, 2022).

The CVE-2017-5638 “allows remote attackers to execute arbitrary commands via a crafted Content-Type” (National Institute of Standards and Technology, 2017). The reason why I chose this CVE is because SNORT simply has to locate "Content-Type:

%{" within the message (National Institute of Standards and Technology, 2017). This CVE was published on 03/10/2017, and last modified on 04/19/2025 (National Institute of Standards and Technology, 2017). This is the rule:

- alert tcp any any -> any 80 (msg:"Apache Struts2 RCE Attempt"; content:"Content-Type: %{"; sid:1000003; rev:1;) (Microsoft CoPilot, 2025).

With CVE-2021-44228, an attacker “can execute arbitrary code loaded from LDAP servers when message lookup substitution is enabled.” (National Vulnerability Database, 2021). If the server is able to send “${jndi:...} “ within a message, the hacker could run code on the server (National Vulnerability Database, 2021). I picked this CVE since it is well known. It was published on 12/10/2021 and last modified on 04/03/2025 (National Vulnerability Database, 2021). This is the rule:

- alert tcp any any -> any 80 (msg:"Log4Shell Exploit Attempt"; content:"${jndi:"; sid:1000002; rev:1;) (Microsoft CoPilot, 2025).

In order to add my rules, I navigated to “local” file within my SNORT file and opened it with Notepad. I first started with inputting the rule for CVE-2021-44228, which was: alert tcp any any -> any 80 (msg:"Log4Shell Exploit Attempt"; content:"${jndi:"; sid:1000002; rev:1;). It thankfully worked, please reference Figures 13 and 14.

Next, I edited the local file to include the rules I created for CVE-2017-5638, which is: alert tcp any any -> any 80 (msg:"Apache Struts2 RCE Attempt"; content:"Content-Type: %{"; sid:1000003; rev:1;). I navigated to “local” file within my SNORT rules, opened it with Notepad, and added the rule in the file. SNORT then ran successfully.

Conclusion

In conclusion, this lab provided me with a valuable experience with using SNORT for this first time. I was able to learn how the tool operates, it's settings and rules, how to edit the rules myself, and how to monitor traffic between two VMs. Altogether, I was able to improve my understanding of intrusion detection, and I am grateful for the opportunity.

Citations

ChatGPT. "Can you explain all the rules in this screenshot?" OpenAI, 7 May 2025, chat.openai.com.

Cyvatar. “Writing Snort Rules with Examples and Cheat Sheet.” CYVATAR.AI, 27 Jan. 2022, cyvatar.ai/write-configure-snort-rules/. Accessed 14 May 2025.

Fortinet. “SNORT—Network Intrusion Detection and Prevention System.” *Fortinet*, 2023, www.fortinet.com/resources/cyberglossary/snort. Accessed 7 May 2025.

“IDS vs IPS – Differences between IDS and IPS | Versa Networks.” *Versa*, versa-networks.com/sd-wan/ids-ips/. Accessed 7 May 2025.

Microsoft CoPilot. "What arguments should I use so that I can write a SNORT rule for CVE-2017-5638 and CVE-2021-44228?" 14 May 2025, https://copilot.microsoft.com/chats/ajmj7eurAaDy2aAmw84mA.

National Institute of Standards and Technology. “NVD - CVE-2017-5638.” *Nvd.nist.gov*, 10 Mar. 2017, nvd.nist.gov/vuln/detail/CVE-2017-5638. Accessed 14 May 2025.

National Vulnerability Database. “NVD - CVE-2021-44228.” *Nvd.nist.gov*, 10 Dec. 2021, nvd.nist.gov/vuln/detail/CVE-2021-44228. Accessed 14 May 2025.

Appendix

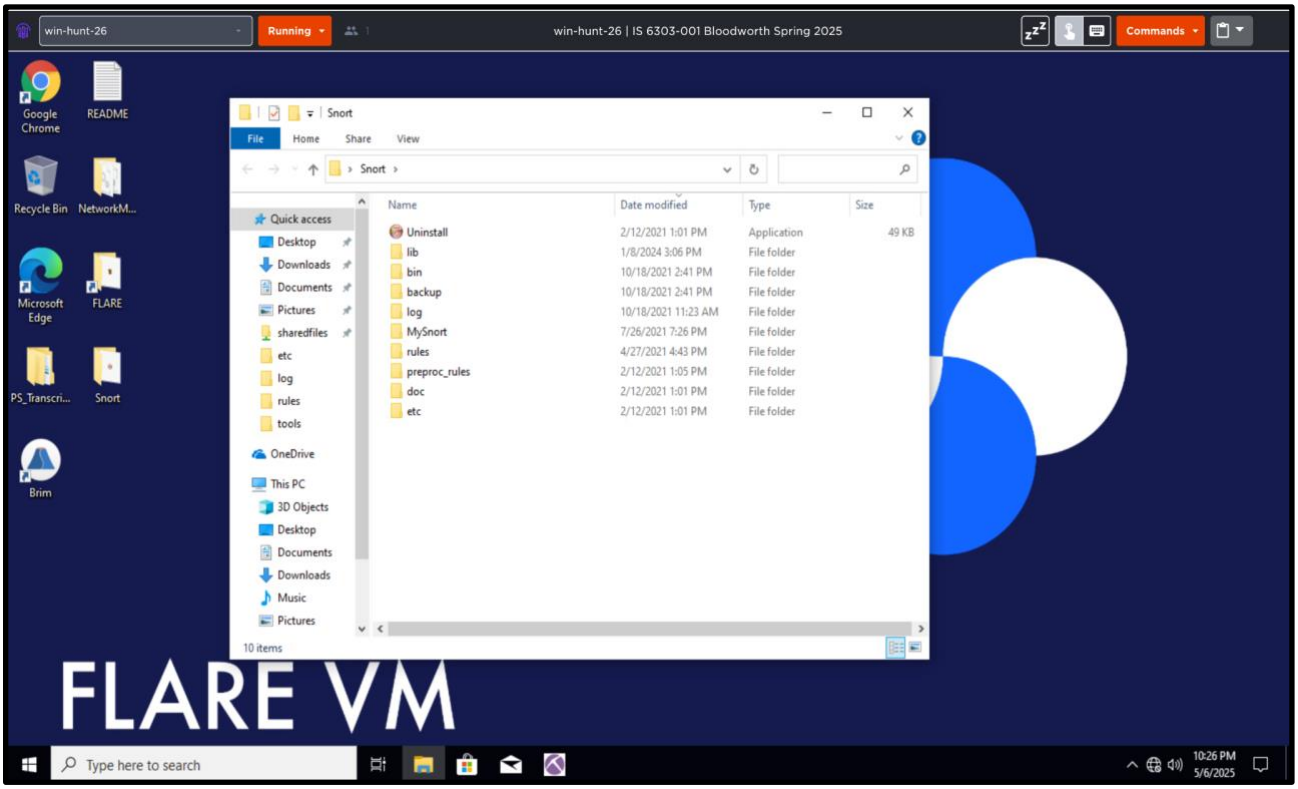

Figure 1: *Screenshot of the SNORT files in Win-Hunt-26.*

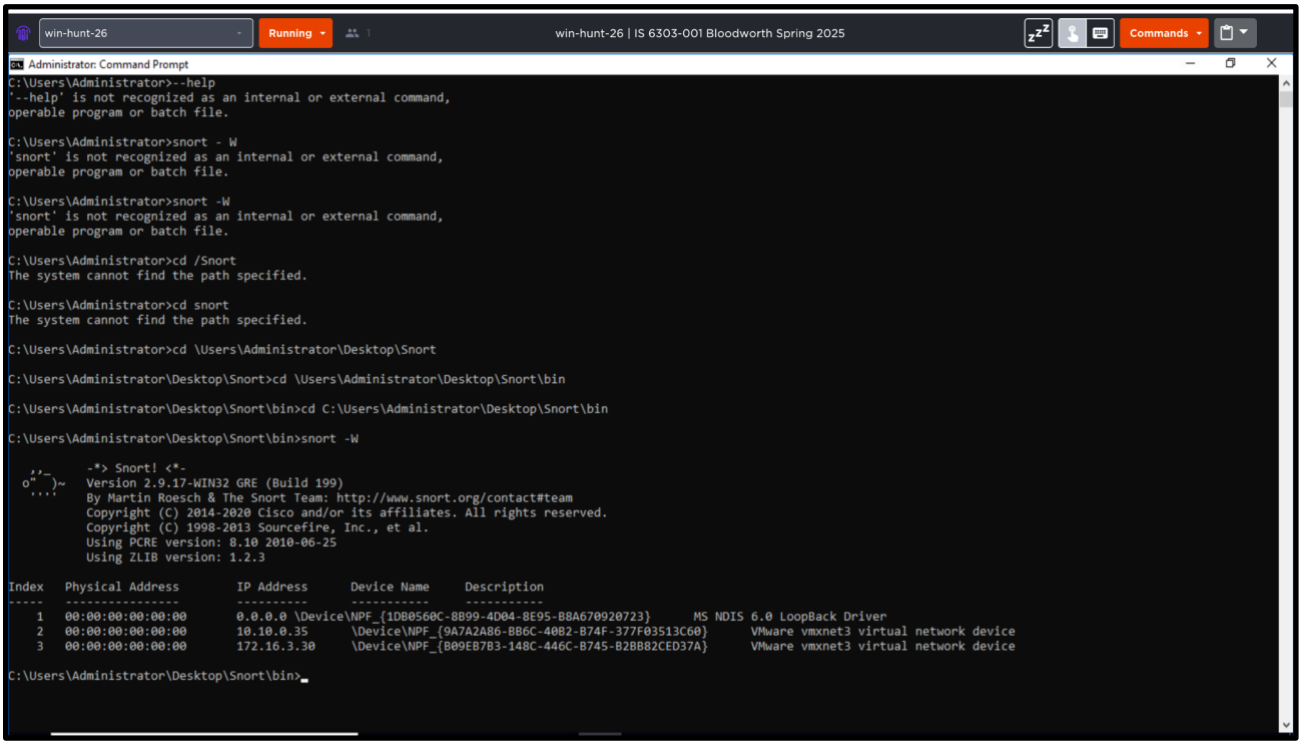

Figure 2: *This includes the command "snort -W".*

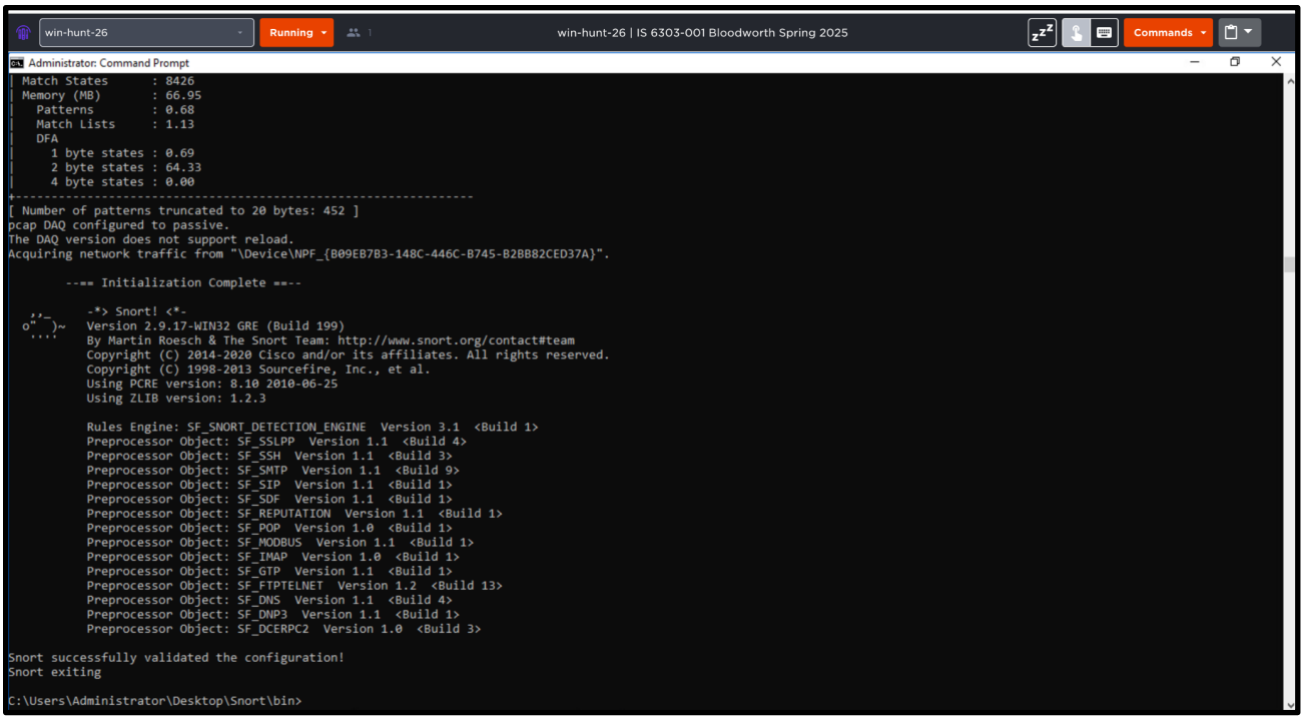

Figure 3: *I entered "snort -W" and then "snort -i 3 -c ..\etc\snort.conf -T" and this was the output. It ran successfully!*

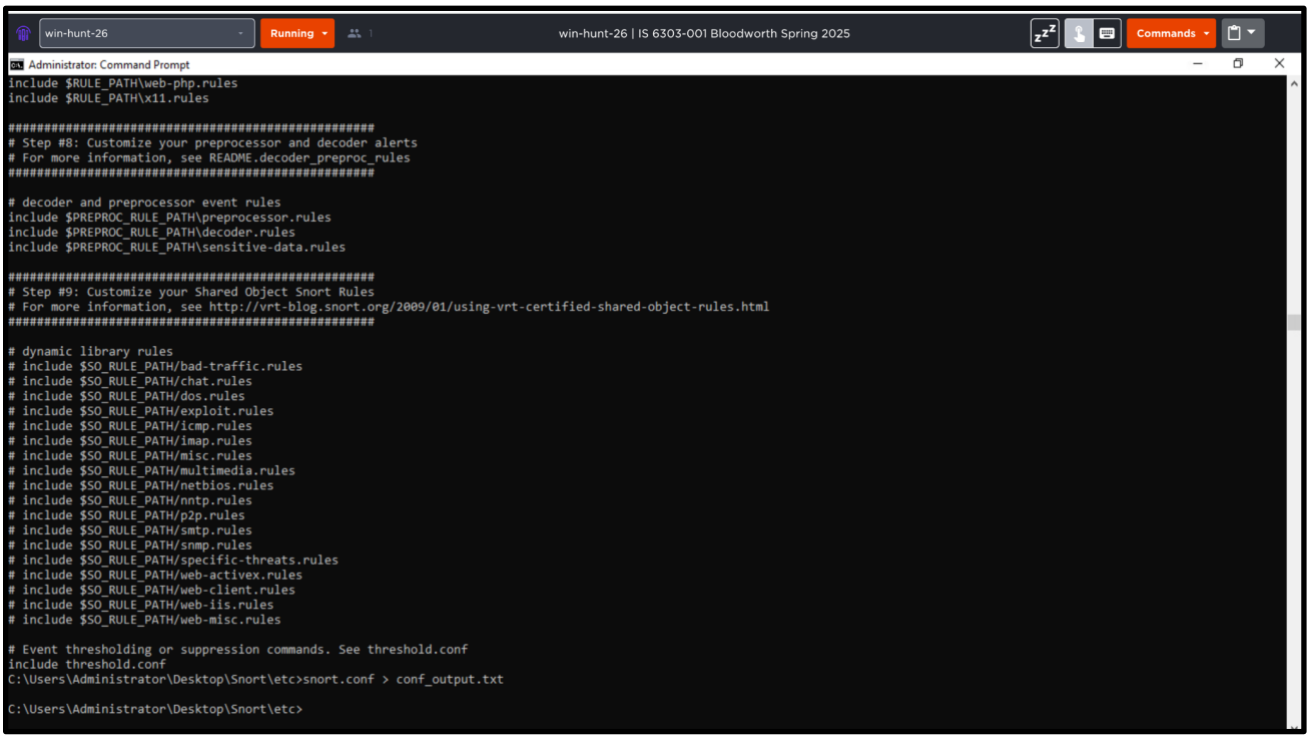

Figure 4: *The Figure above includes the commands "type snort.conf" and "type snort.conf > conf_output.txt"*

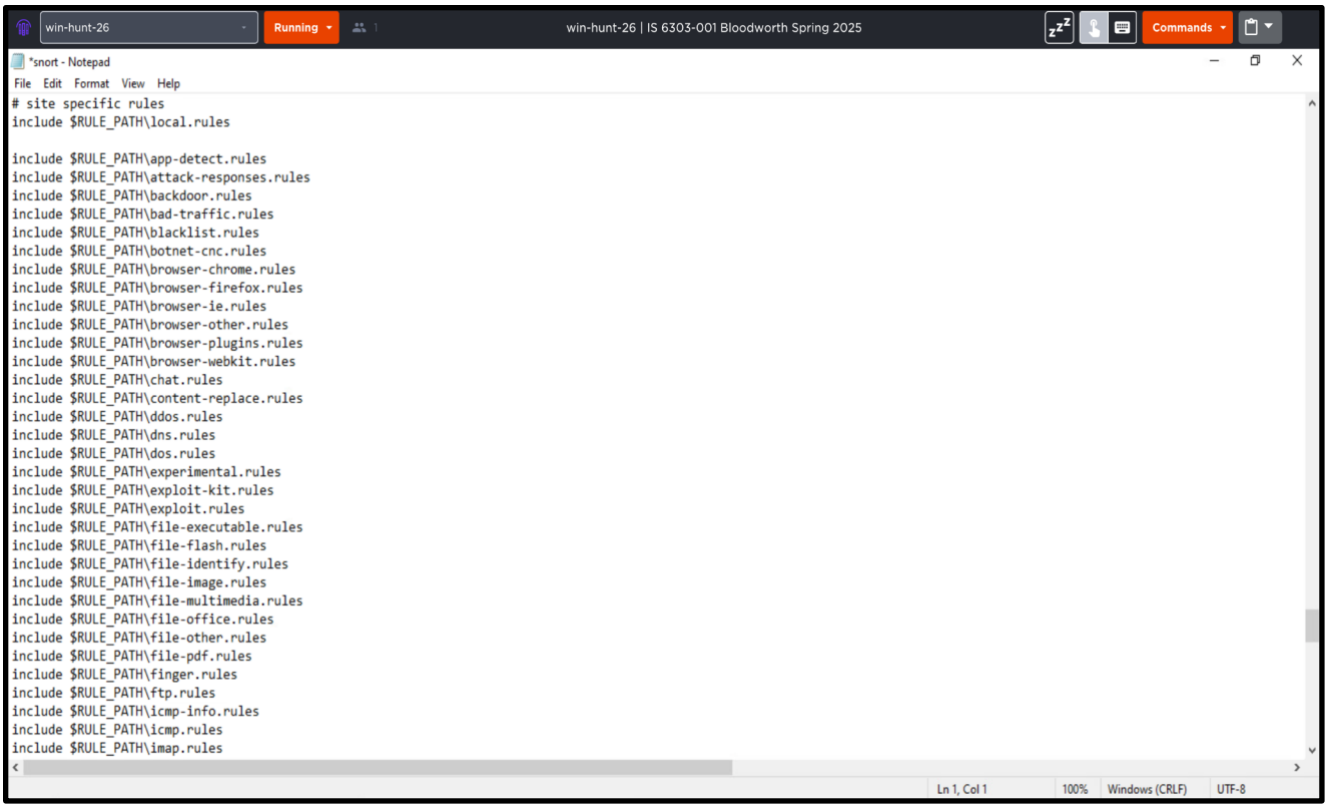

Figure 5: *Above is the file I created called "conf_output.txt". The rules are located here.*

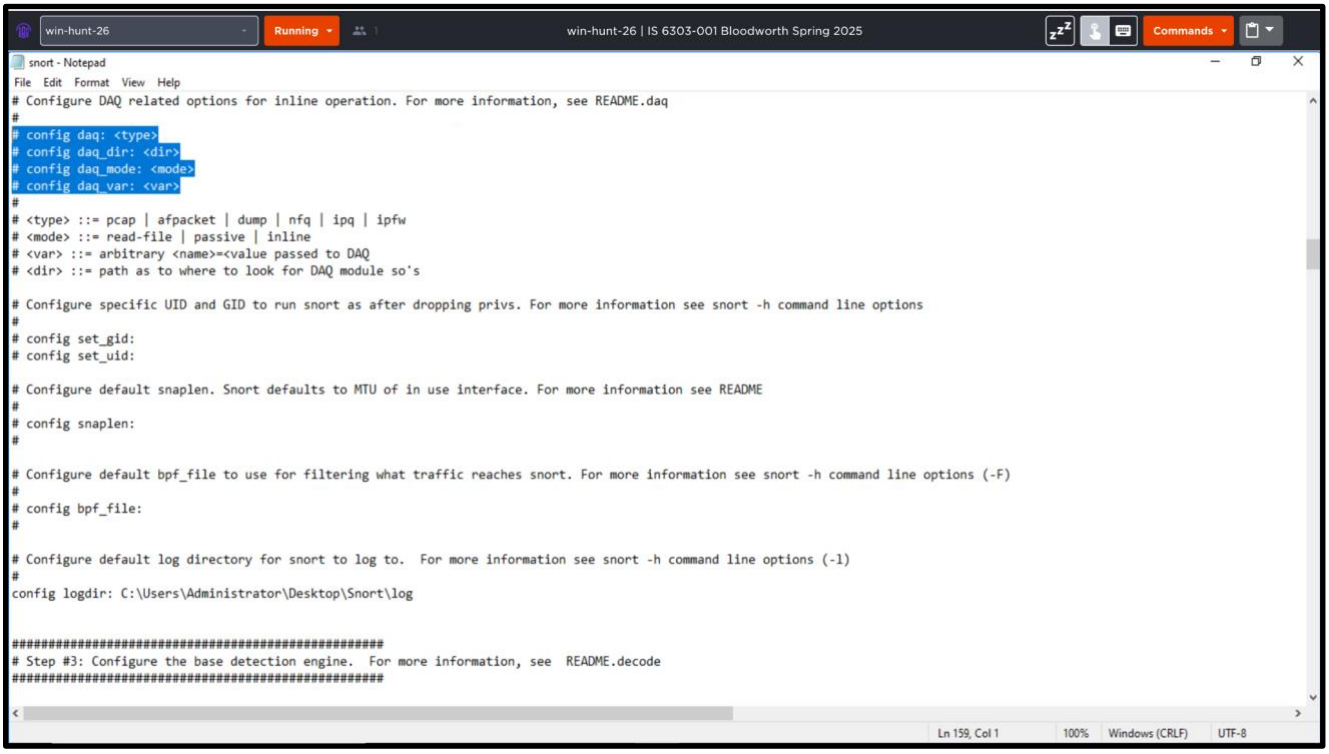

Figure 6: *Above is the options within the "conf_output.txt" file.*

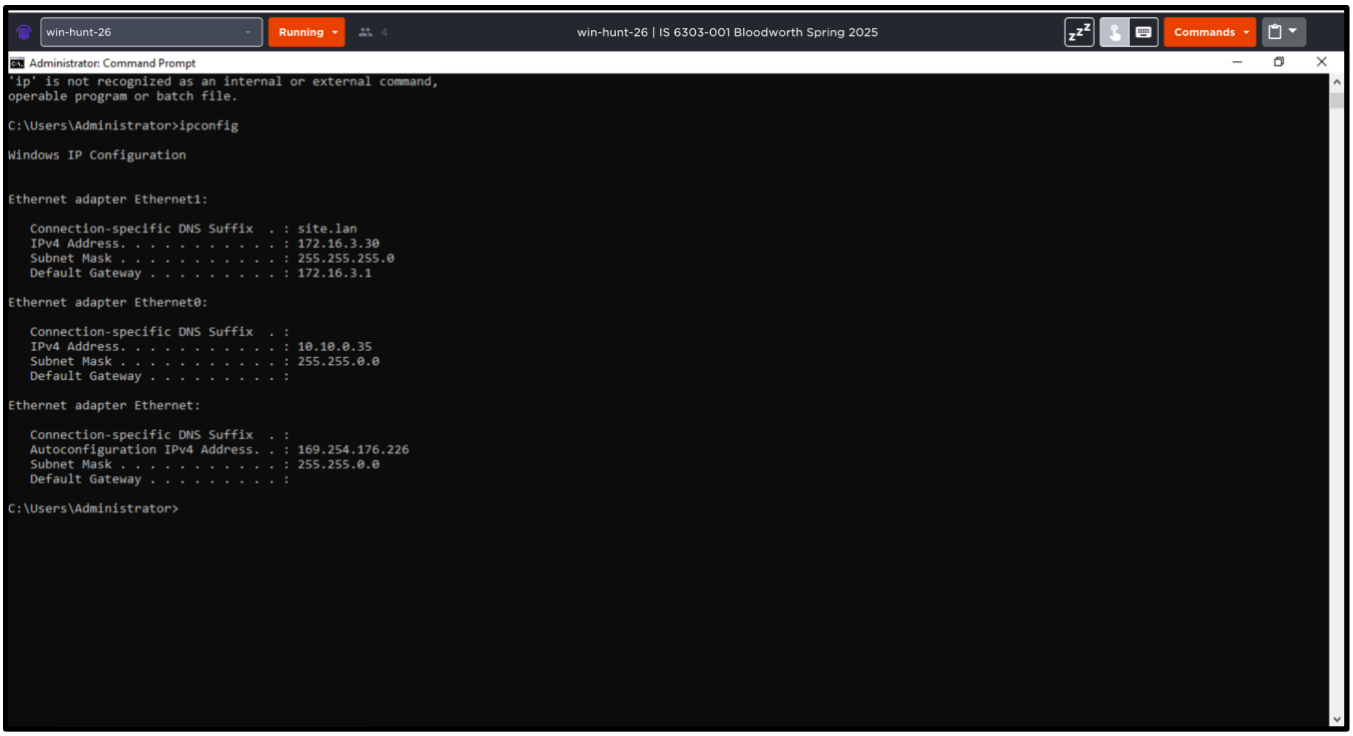

Figure 7: *Entering "ipconfig" on my Win-Hunt-26.*

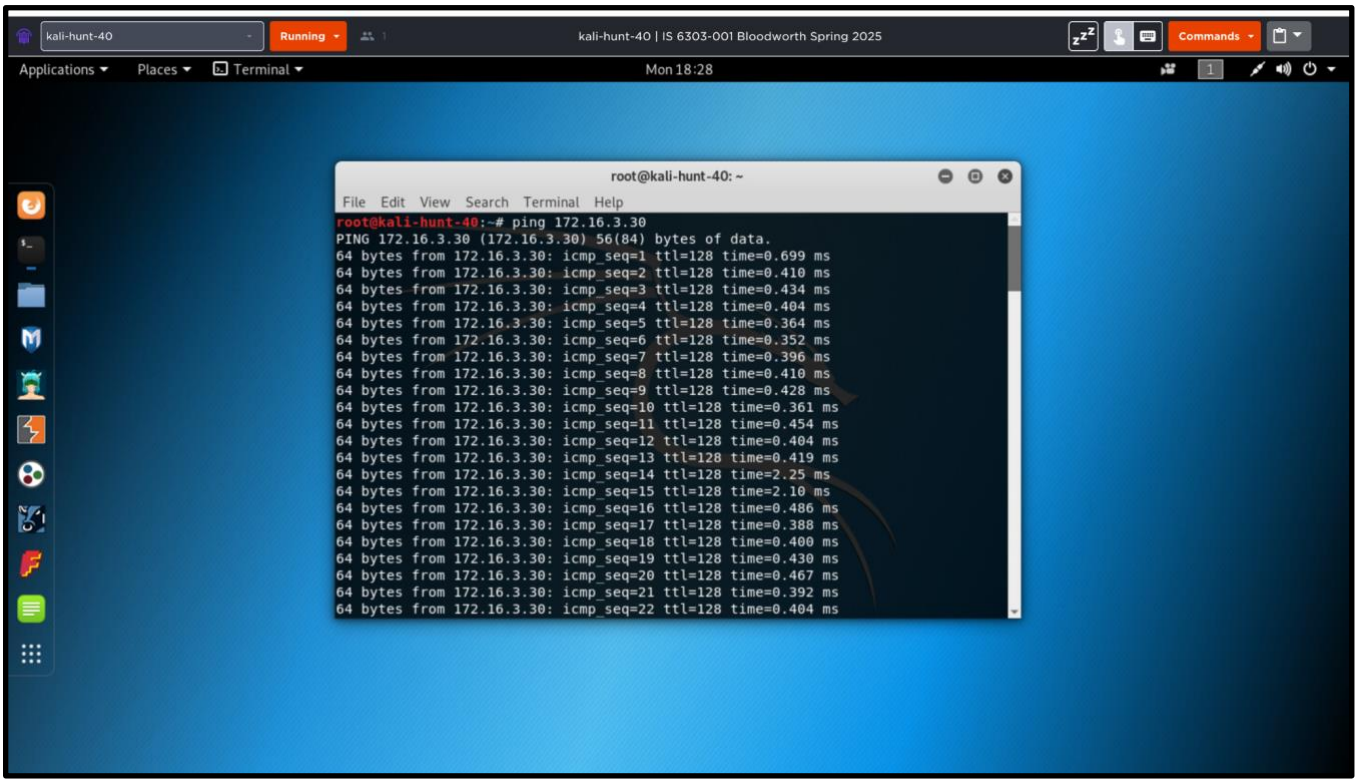

Figure 8: *Pinging Win-Hunt-26 from Kali-Hunt-40.*

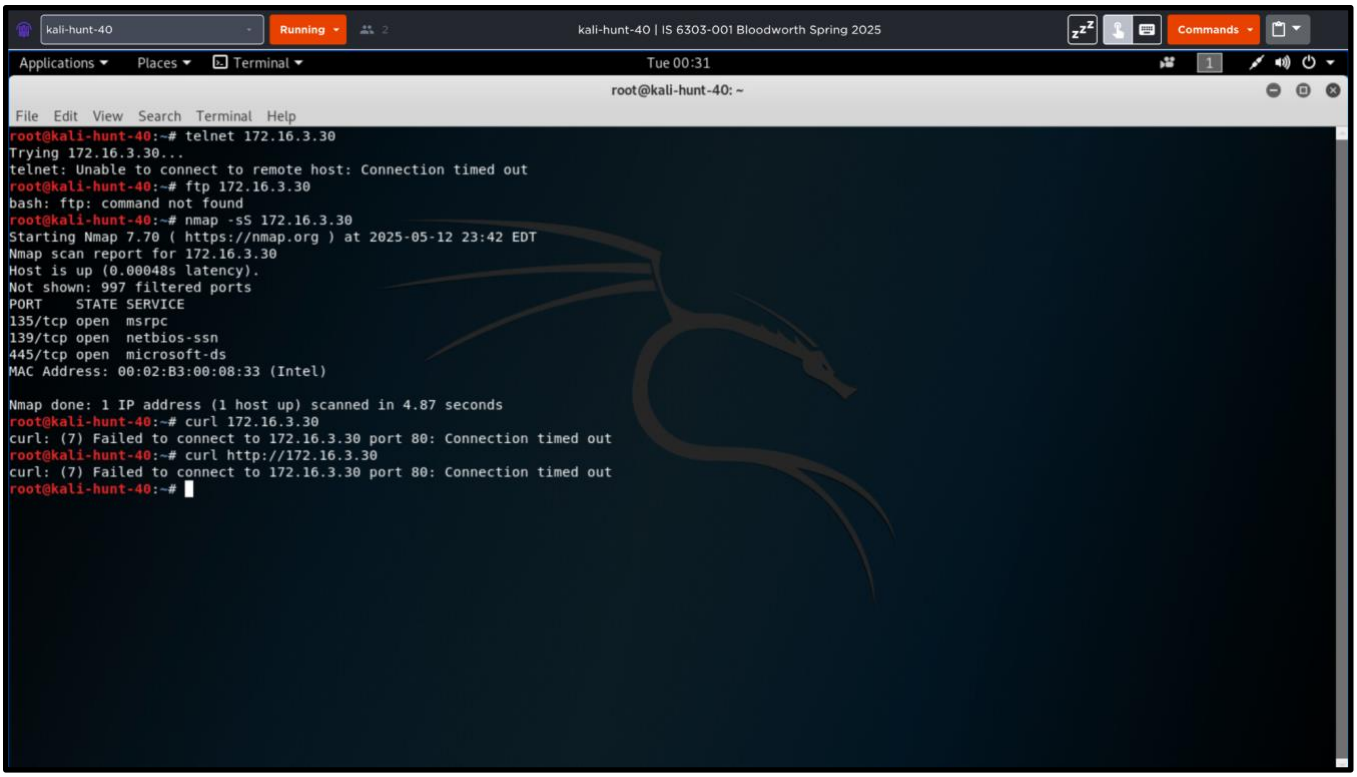

Figure 9: Above is the screenshot of the commands I ran: telnet, ftp, nmap -sS, and curl in Kali-Hunt-40.

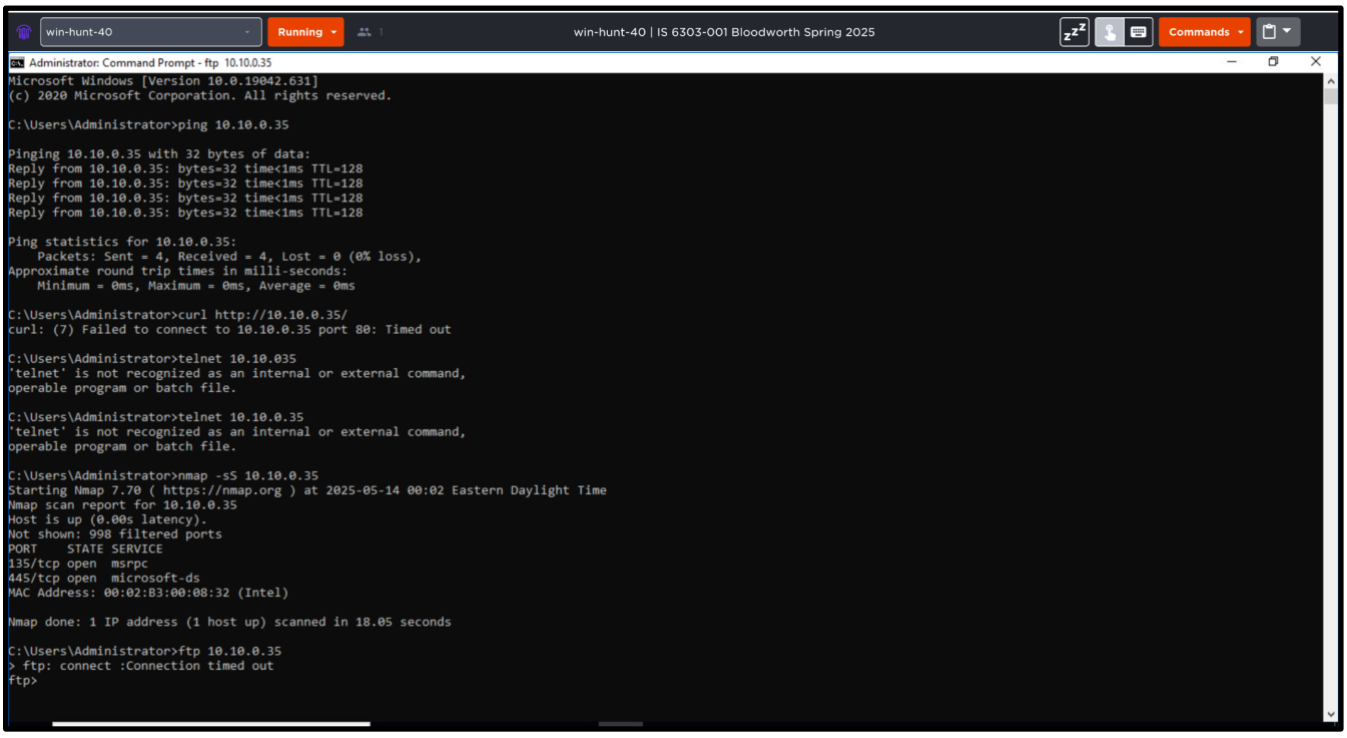

Figure 10: Above are the commands I ran ping, curl, telnet, and nmap -sS in Win-Hunt-40.

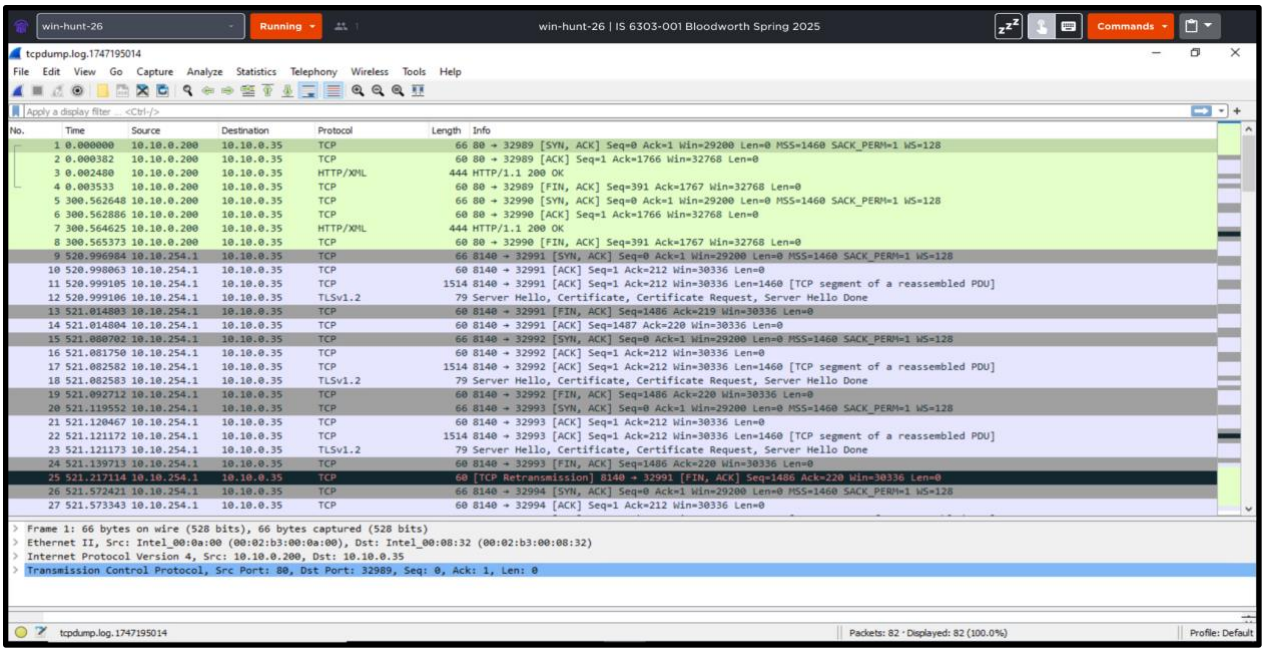

Figure 11: The photo above is the file that I opened on Win-Hunt-26 in Wireshark after running various commands from Win-Hunt-40.

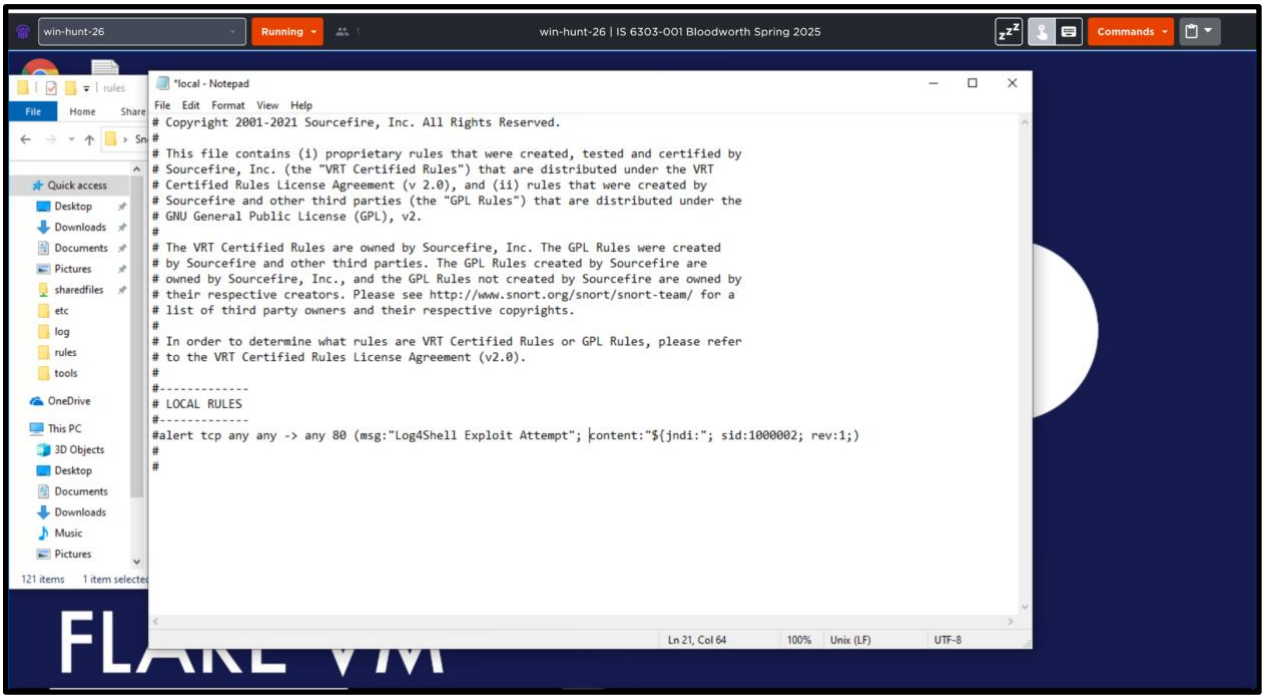

Figure 12: *Adding "alert tcp any any -> any 80 (msg:"Apache Struts2 RCE Attempt"; content:"Content-Type: %{"; sid:1000003; rev:1;)" to the local files for* SNORT.

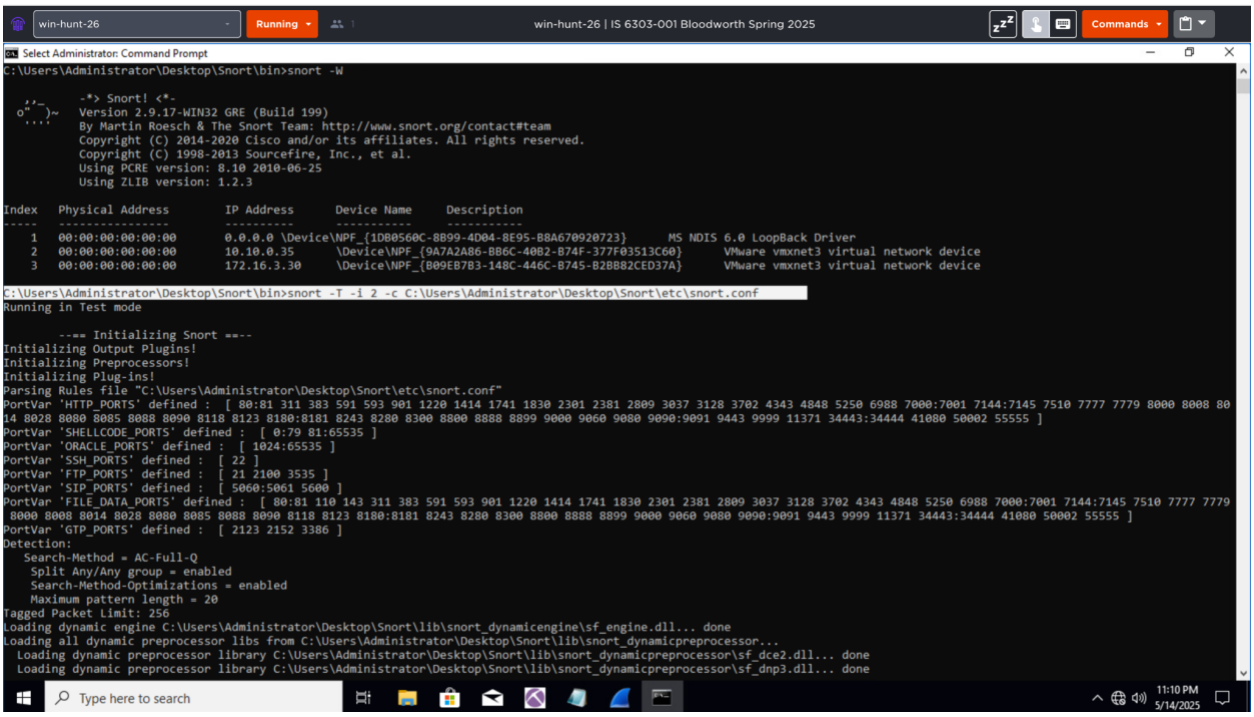

Figure 13: Above is the screenshot of the command entered whilst running SNORT after updating the local rules.

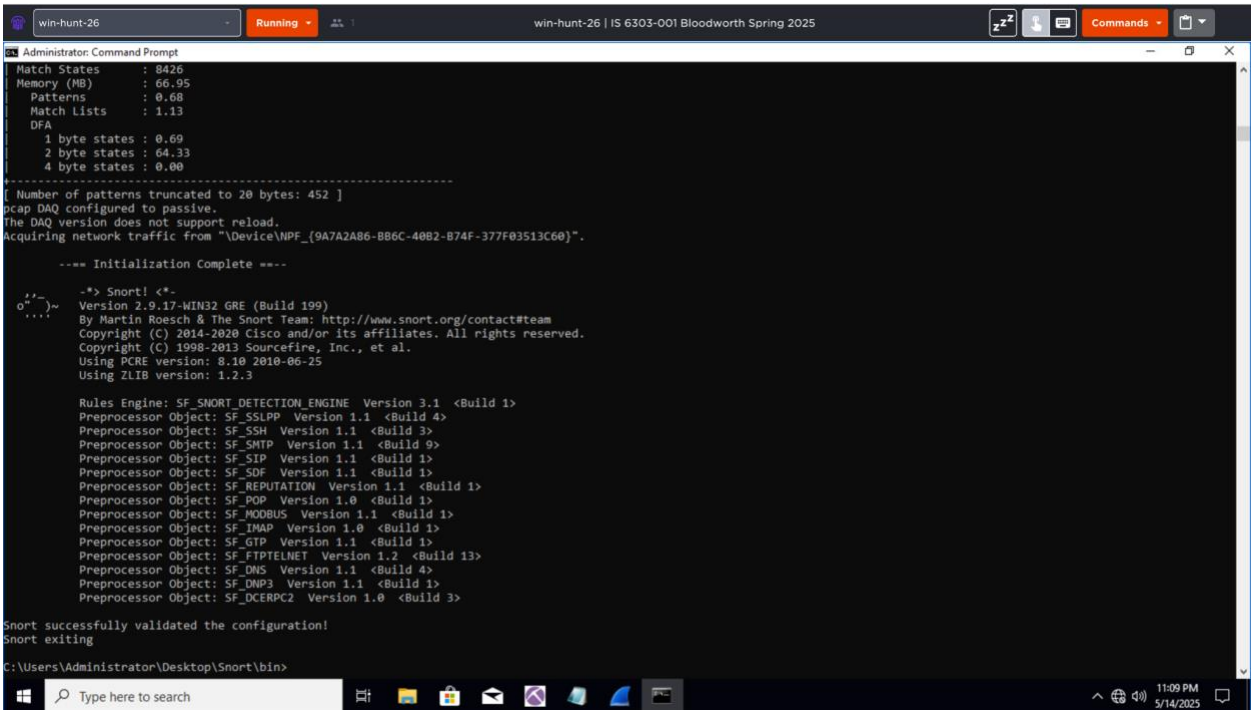

Figure 14: *Above is the screenshot of SNORT running successfully after editing the local rules for CVE-2017-5638.*

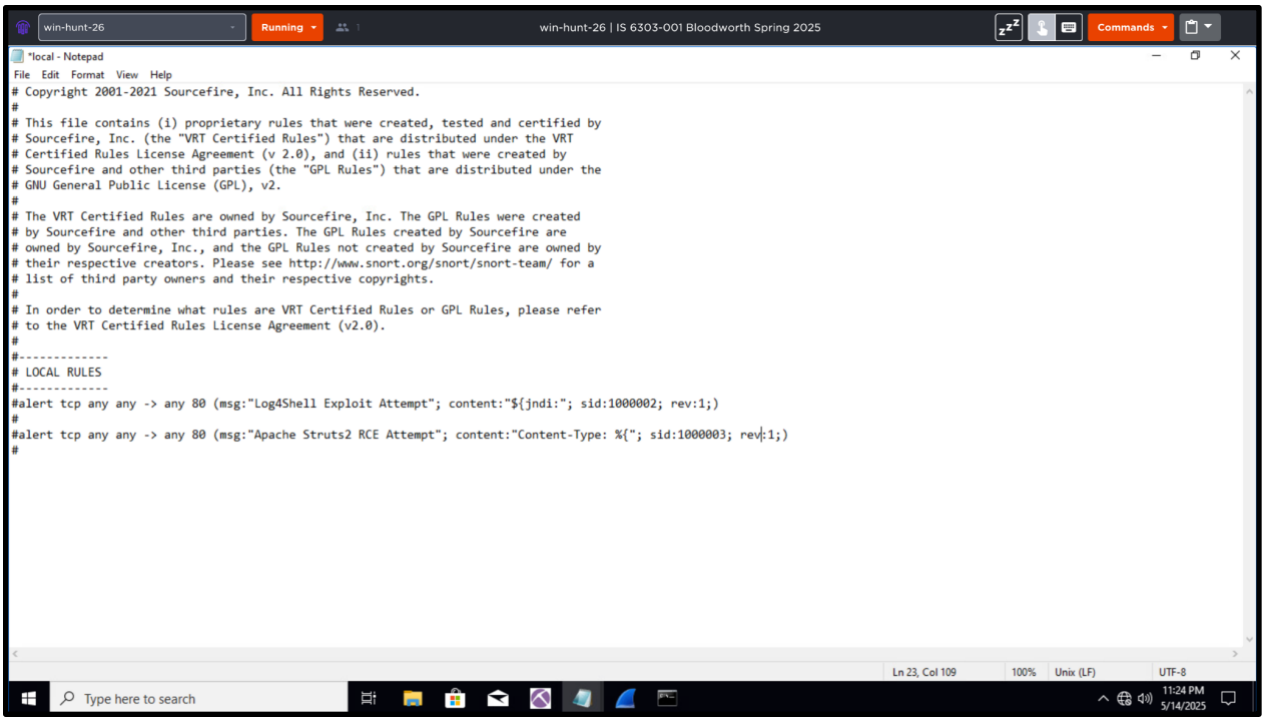

Figure 15: *Editing the local file for CVE-2017-5638: "Apache Strut2".*

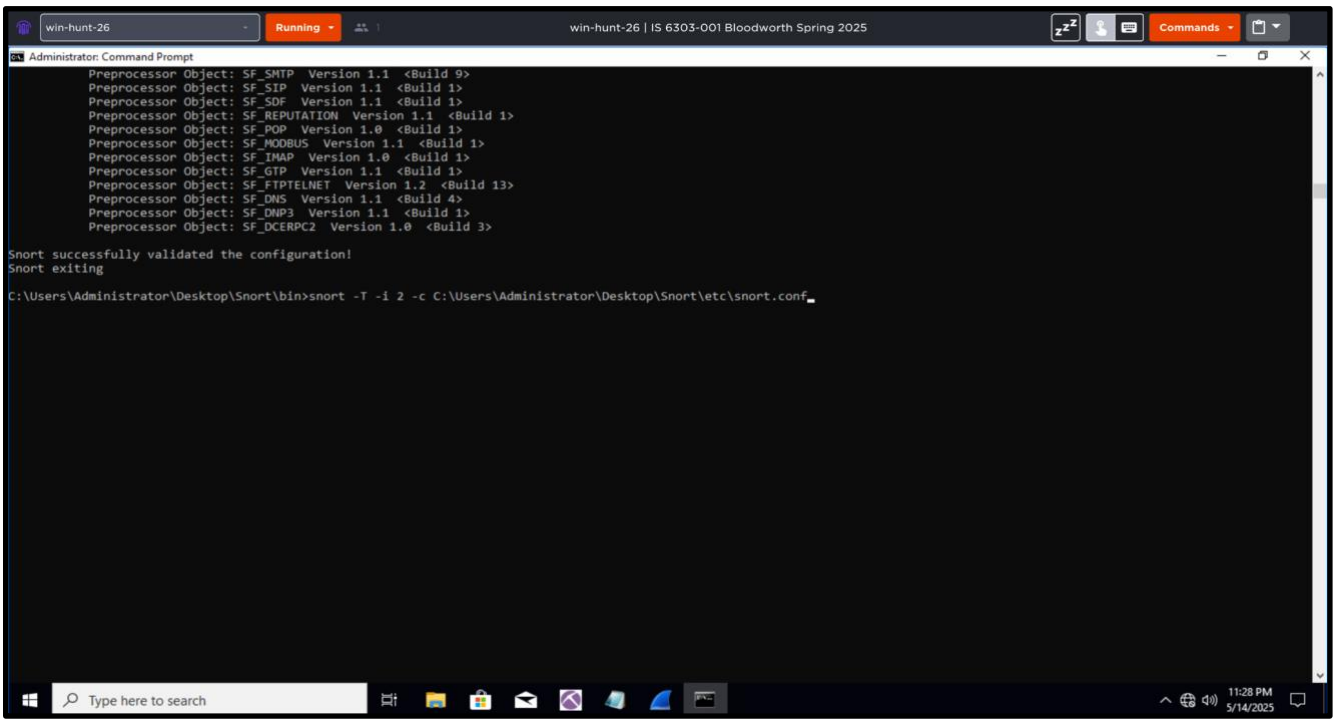

Figure 16: *I ran SNORT once again after adding the CVE-2017-5638 rule.*

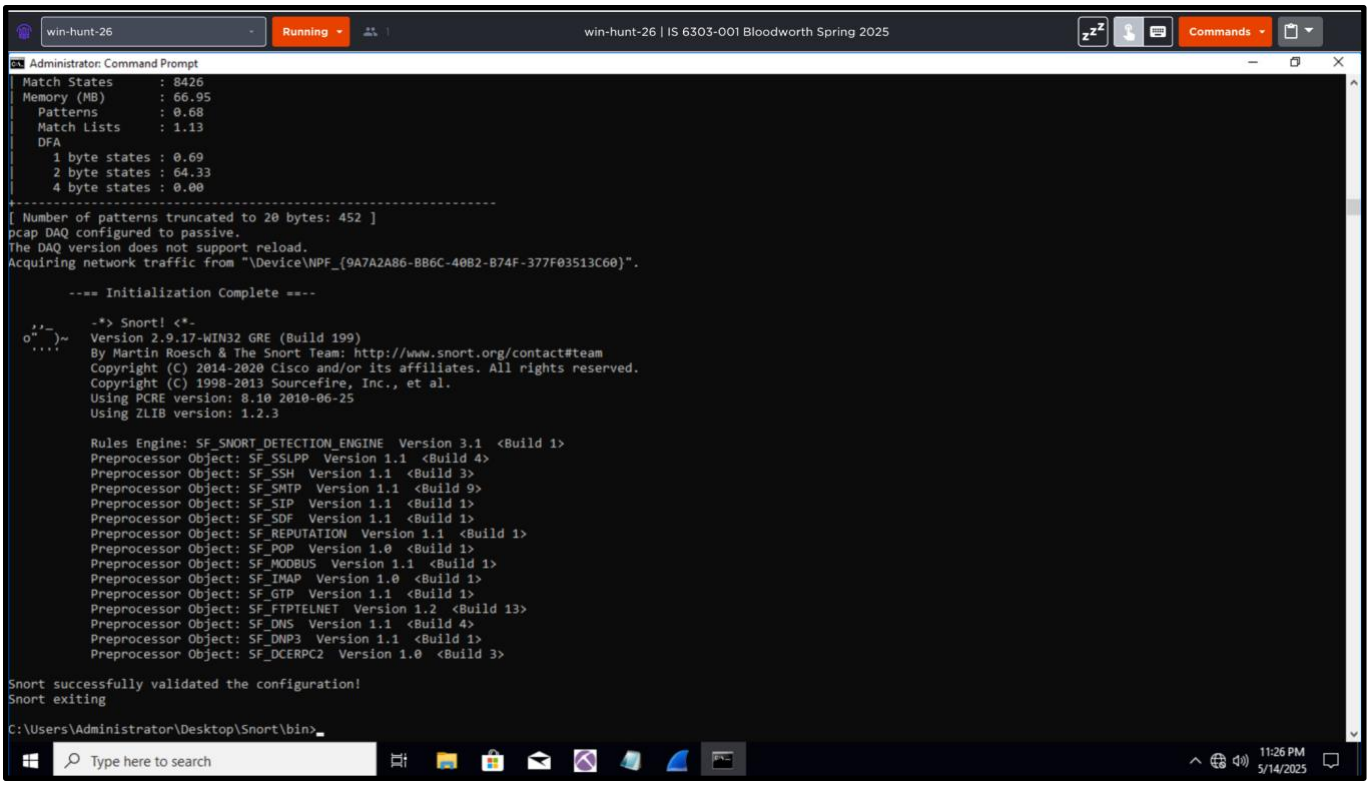

Figure 17: *I ran SNORT once again after adding the CVE-2017-5638 rule and it was a success.*